



TP Tolerancia:

Money Laundering Analysis

Integrante	Padrón
Carbajal Robles Kevin Emir	108304
Rea Matias	99770
Sendra Alejo	107716

Sistemas Distribuidos I (TA050)
Cátedra Roca - 1er cuatrimestre de 2026

25 de Junio de 2026

Índice

Índice.....	1
Introducción.....	2
Diseño.....	2
Procesamiento y flujo de datos.....	3
Procesamiento de datos.....	3
Diagrama de robustez - Escenarios.....	5
Consulta 1.....	5
Consulta 2.....	5
Consulta 3.....	6
Consulta 4.....	6
Consulta 5.....	7
Comunicación y comportamiento.....	8
Consulta 3.....	8
Consulta 4.....	9
Flujo del sistema.....	11
Punto de Entrada.....	11
Consulta 3.....	11
Consulta 4.....	12
Organización Modular y Estructura.....	13
Diagrama de paquetes.....	13
Infraestructura y Despliegue.....	15
Tolerancia a fallos problemáticas y soluciones.....	16
Nodos y comportamiento ante fallos.....	16
Identificación y Manejo de Duplicados.....	16
Persistencia de Datos y Recuperación.....	17
WatchDog y algoritmo de elección (Bully).....	17
Consideraciones.....	17
Referencias.....	18

Introducción

El presente informe detalla el seguimiento, las modificaciones y detalles de implementación del diseño propuesto en el hito anterior. En este caso se busca que el sistema sea tolerante a fallas, por lo cual, la caída de procesos de forma aleatoria no debe afectar los resultados de la ejecución del sistema.

Para el desarrollo de esta característica se consideraron los siguientes supuestos:

1. Toda instancia caída del sistema eventualmente se volverá a levantar por el sistema de WatchDogs.
2. Por lo menos, habrá 1 instancia de WatchDogs activa.

Diseño

El diagrama general de robustez del sistema se mantiene igual, en el aspecto general, pero se ha agregado otro conjunto de workers con fines de monitorear, y gestionar, todos los nodos.

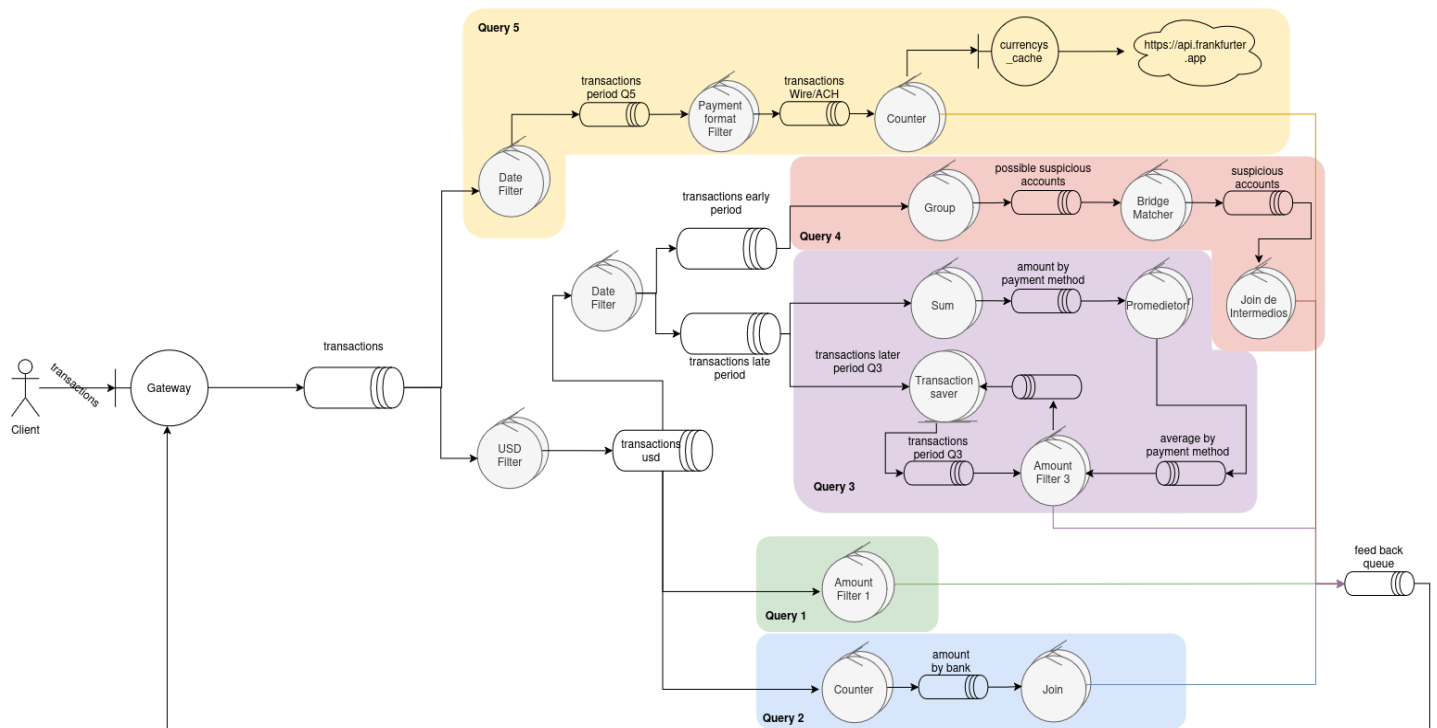


Figura 1.1: Diagrama de Robustez general.

Notar, en la Figura 1.2, el diagrama de robustez que incluye al nuevo grupo de workers y la comunicación del mismo con el resto de los nodos del sistema.

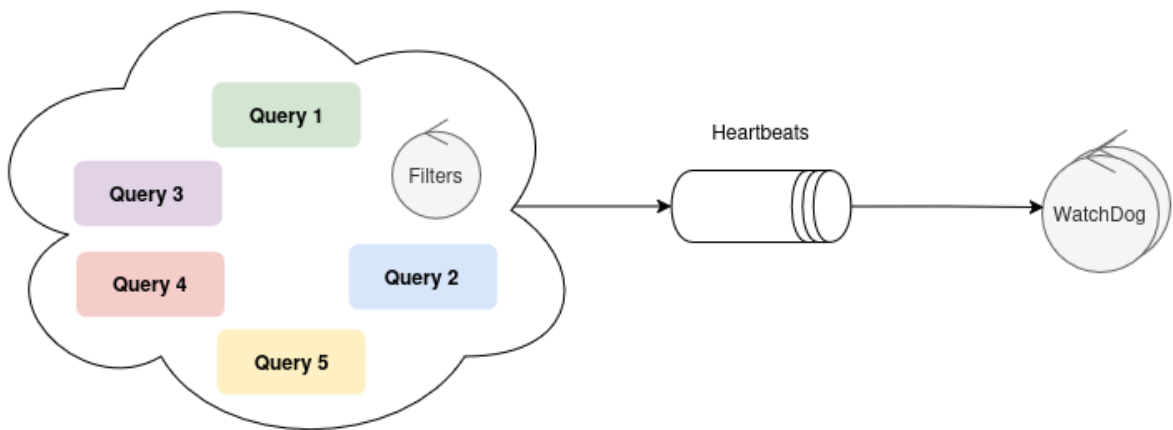


Figura 1.2: Diagrama de Robustez de Watchdogs.

Procesamiento y flujo de datos

En esta sección se tiene el objetivo de desarrollar sobre la implementación del sistema, explicando la cadena de procesos por la que pasa la información, así como la comunicación entre el cliente y el sistema, y la comunicación interna.

Procesamiento de datos

A continuación, se detalla como es el procesamiento de datos para cada consulta.

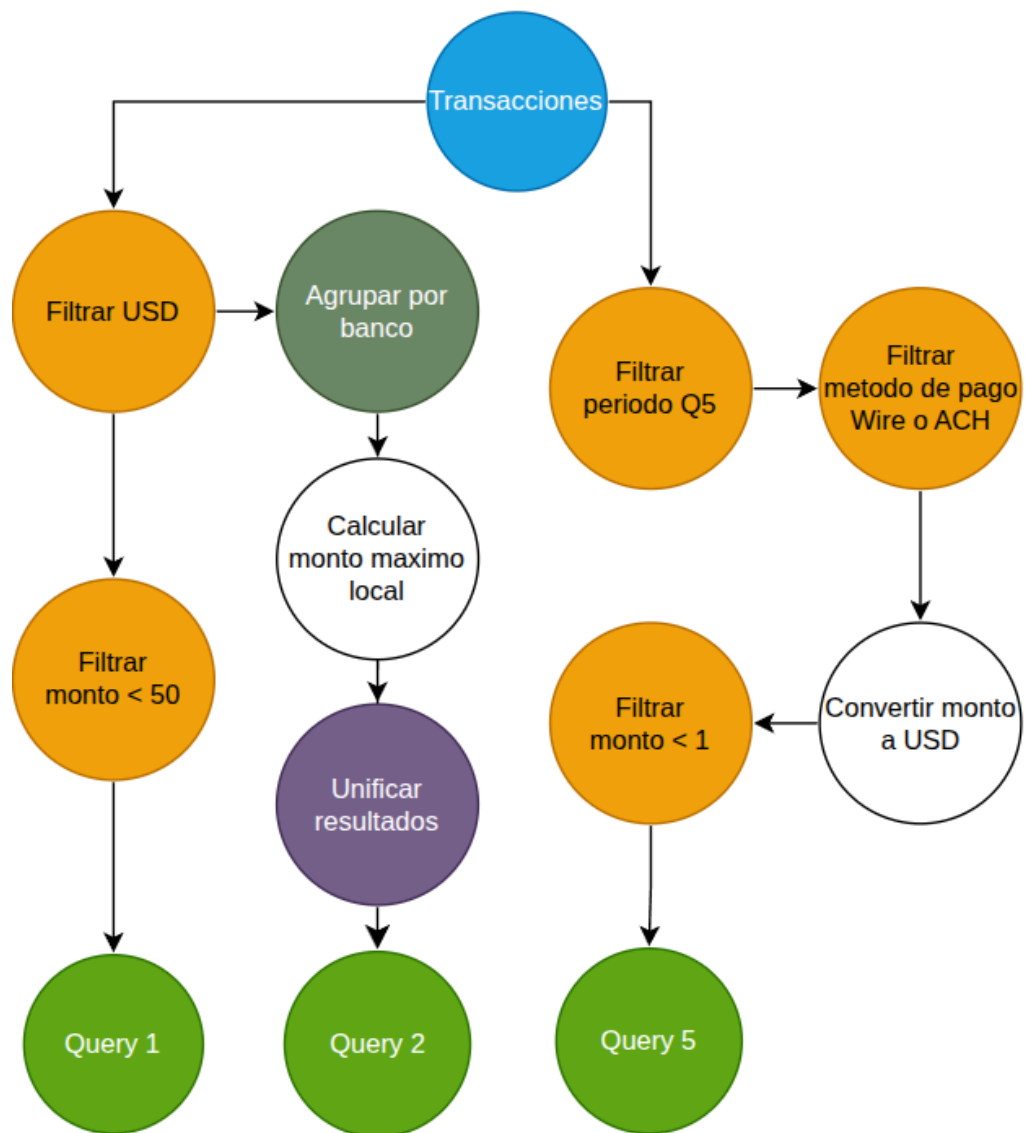


Figura 2.1: DAG para las consultas 1, 2 y 5

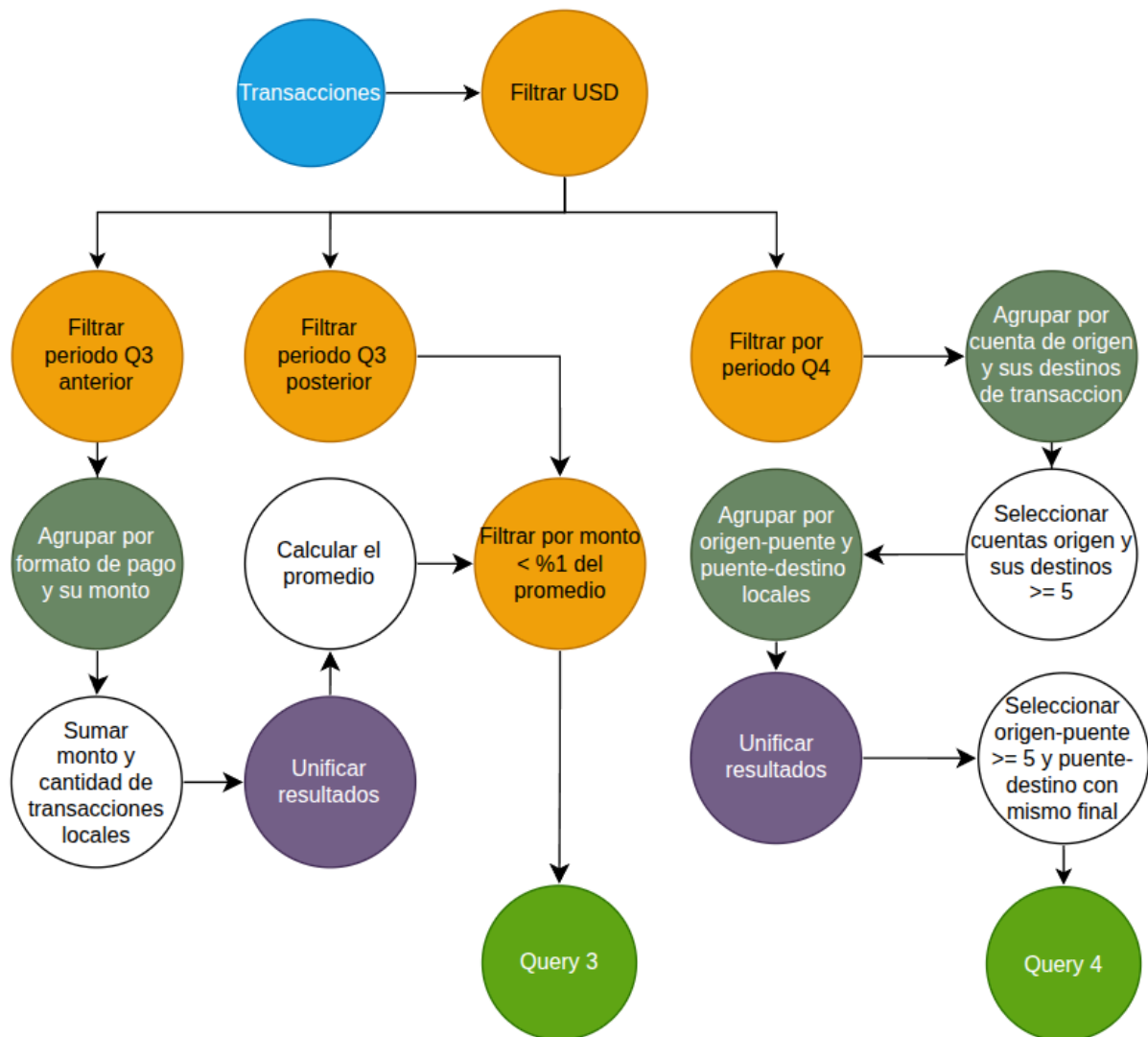


Figura 2.2: DAG para las consultas 3 y 4

En el diagrama se observa que el flujo principal de datos atraviesa una serie de filtros comunes, tales como el filtro de moneda USD y la segmentación por periodos temporales. A partir de estas etapas iniciales, las transacciones se distribuyen a través de diversos nodos especializados en tareas de agrupación, mapeo y agregación de datos.

Si bien el sistema optimiza el procesamiento paralelo, algunos flujos presentan una complejidad lógica superior debido a sus requerimientos:

Algunos flujos pueden resultar complejos a simple vista, como por ejemplo:

- Consulta 3: Requiere que el sistema calcule un promedio global por formato de pago en un periodo inicial. Los nodos de procesamiento posterior permanecen a la espera de este valor para habilitar el filtrado final.
- Consulta 4: Se identifica primero cuentas orígenes con una dispersión hacia 5 o más destinos. Una vez marcados como sospechosos, el sistema rastrea los movimientos de estos intermediarios para confirmar si convergen en un destino final común
- Consulta 5: Se integra un nodo de consumo de API externa. Este componente realiza la conversión de cada divisa descubierta a USD, permitiendo que la validación de umbrales sea independiente de la moneda original de la transacción.

Cada flujo de procesamiento concluye en un nodo sumidero específico para cada consulta, donde los resultados se consolidan y se retornan al Gateway para su entrega al cliente.

Diagrama de robustez - Escenarios

El punto de entrada al sistema es el Gateway, componente encargado de recibir y canalizar el flujo de transacciones enviadas por el cliente. Inicialmente, todas las operaciones siguen un flujo común de ingesta y pre-procesamiento:

- Ingesta: El cliente envía lotes de transacciones al Gateway.
- Despacho: Las transacciones se dirigen hacia un exchange desde el cual todos los nodos interesados reciben la totalidad de las transacciones enviadas por el cliente.

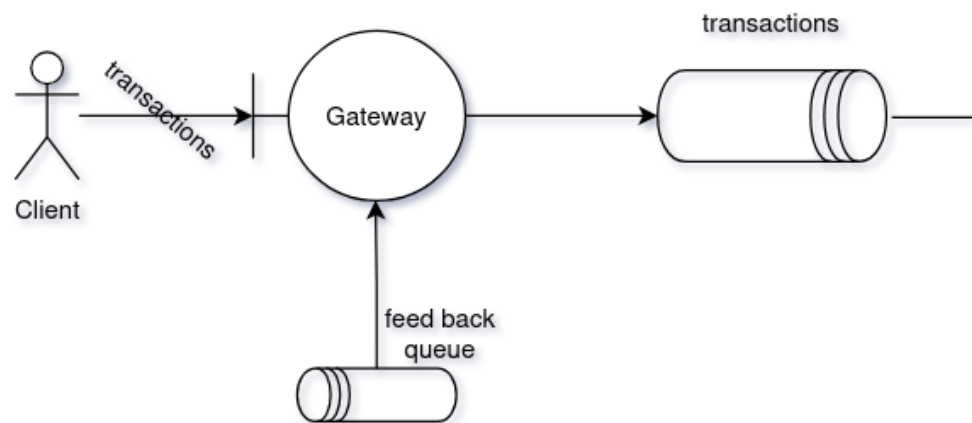


Figura 3.1: Flujo inicial de todas las consultas

Consulta 1

Para la primera consulta, el sistema identifica las transacciones realizadas en dólares con un monto inferior a 50. Una vez filtradas, la cuenta de origen, el destino y el monto se retornan al Gateway a través de una cola de mensajes para su consolidación final. Una vez un worker “Amount Filter 1” recibe aviso del EOR, por parte del usuario,

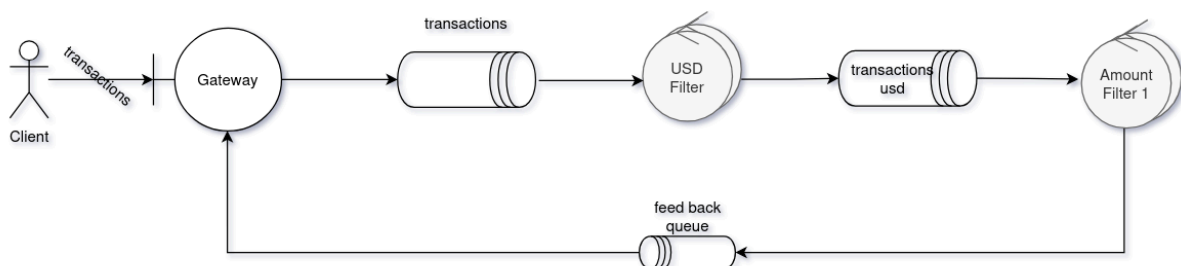


Figura 3.2: Diagrama de robustez de la consulta 1

Consulta 2

En este escenario, se filtran las transacciones en USD para luego agruparlas por entidad bancaria. Se calcula el monto máximo local en cada nodo de procesamiento Counter. Una vez que el cliente haya terminado de enviar, se unifican los resultados parciales en el Join para determinar el máximo global por banco, devolviendo la información (Banco, Cuenta y Monto) al Gateway.

```

sequenceDiagram
    actor Filter
    participant transactions as transactions
    actor Counter
    participant join_transactions as join transactions
    actor Join
    participant Gateway
    Filter->>transactions: Payment Format == "US Dollar"
    transactions->>Counter
    Counter->>join_transactions
    join_transactions->>Join
    Join->>Gateway
    Gateway-->>Join: From Bank Account Amount Paid
  
```

Consulta 3

La consulta se divide en dos flujos, donde el segundo depende del primero. Los filters iniciales filtran las transacciones en USD, luego por periodos.

1. Periodo 1/9/2022 al 5/9/2022: se suman los montos por tipo de transacción y la cantidad. Luego el nodo join “average” calcula el promedio de cada una y le envía los resultados al filter por monto.
2. Periodo 6/9/2022 a 15/9/2022: mientras no tenga los resultados de los promedios el filter por monto, transactions saver guardan las transacciones en disco. Al recibir la notificación de los promedios por parte del filter por monto, procede a enviar las transacciones para su post filtrado en función que no superen el centésimo de su promedio por tipo de pago. Estas transacciones que pasan el filtro final son enviadas al gateway mediante feedback queue

Consulta 4

El flujo comienza obteniendo las transacciones que se realizaron en el periodo 1-9-22 al 5-9-22 y sean en USD.

Luego se las agrupan por (origen - n cuentas destino) y (destino - n cuentas origen), de esta forma creamos los patrones Fan-out y Fan-in. Cada grouper contendrá toda la información para de un origen o destino en particular ya que el envío depende del hash de estos valores.

El Bridge matcher comprueba si se cumple un camino desde una cuenta A a una B por distintas cuentas intermedias.

El joiner recopila los resultados de los distintos nodos y lo envía al gateway

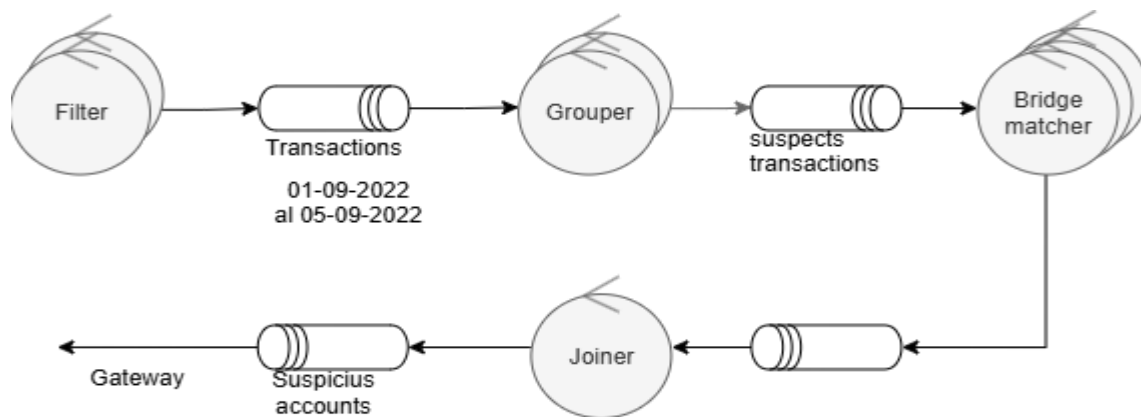


Figura 3.5: Diagrama de robustez de la consulta 4

Consulta 5

Para la quinta consulta, los Filters detectan inicialmente aquellas transacciones entre los periodos [2022-09-01, 2022-09-05] y con método de pago "Wire" o "ACH". Luego, filtran aquellas con montos menores a 1 USD. Debido a la variedad de divisas en el dataset, cada filter se comunica con una API externa para obtener las cotizaciones, y convertir el umbral de "1 USD" a la moneda correspondiente (quote) de la transacción.

Para minimizar las peticiones de red, los nodos de filtrado implementan un mecanismo de caché local de conversiones. De esta manera, si una cotización ya fue consultada por algún nodo, los demás pueden reutilizar dicha información para procesar transacciones similares.

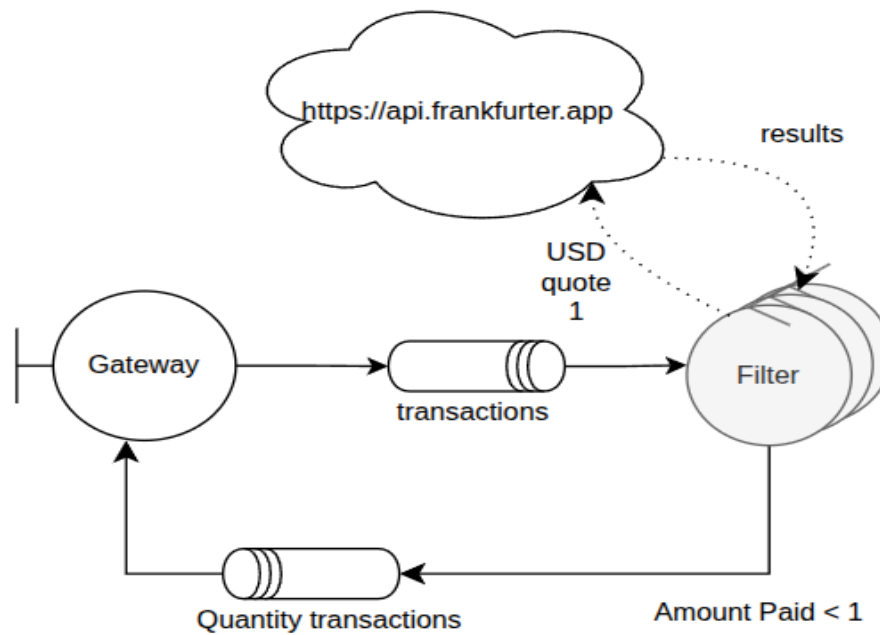


Figura 3.6: Diagrama de robustez de la consulta 5

Control de caídas

Ante la posibilidad de caída de cualquier worker se implementó un grupo de workers basados en el patrón watch dog para monitorear y levantar aquellos workers caídos. Asimismo, ante la posibilidad de que más de un watchdog busque levantar al mismo worker, se implementó un algoritmo de elección de líder para definir al Watchdog encargado de volver a levantar a cualquiera de los nodos caídos, incluidos los mismos Watchdogs.

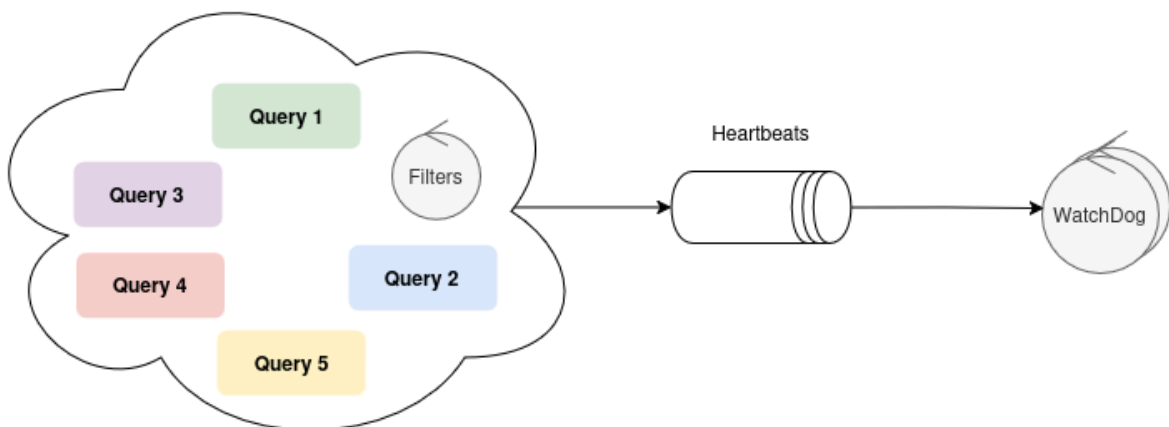


Figura 3.4: Diagrama de robustez del sistema de control de caídas.

Comunicación y comportamiento

Para mayor comprensión sobre cómo se comunican los nodos, y cómo estos se comportan, en la presente sección se desarrollan algunos diagramas para ilustrar su comportamiento.

En el caso general todas consultas comienzan como ilustra el siguiente diagrama:

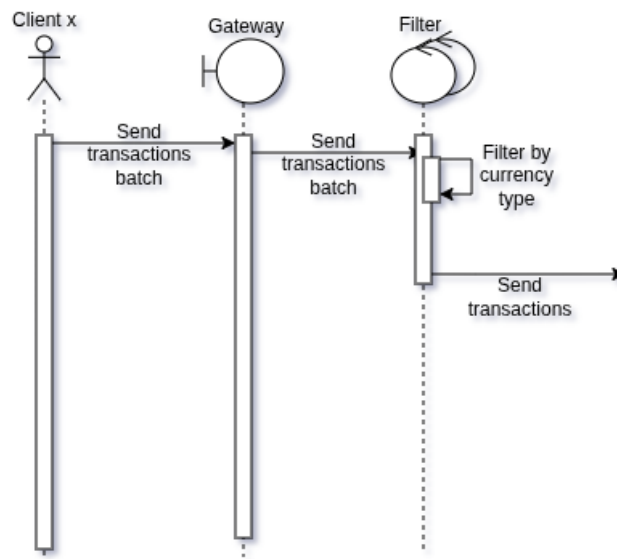


Figura 4.1: Diagrama de secuencia ilustrando comienzo general de las secuencias.

Como se observa en el diagrama 4.1 , toda transacción será enviada al filter y este mismo se encargará de publicar aquellas transacciones en el topic pertinente de su exchange de salida, ya sea para ser considerada en los conteos o como sospechosa ante el cumplimiento de los patrones anteriormente indicados.

A continuación se desarrollan los diagramas de secuencia de los casos más complejos a resolver por el sistema.

Consulta 3

Como se observa en la figura, el filter, además de filtrar por el tipo de moneda se encargará de encolar las consultas según si dicha transacción cumple con las fechas estipuladas por lo impuesto en la consulta 3 o 4.

Posteriormente al filtrado, se ha considerado persistir en disco la información de las transacciones más actuales dado que se requiere el promedio de las transacciones más antiguas para procesarlas.

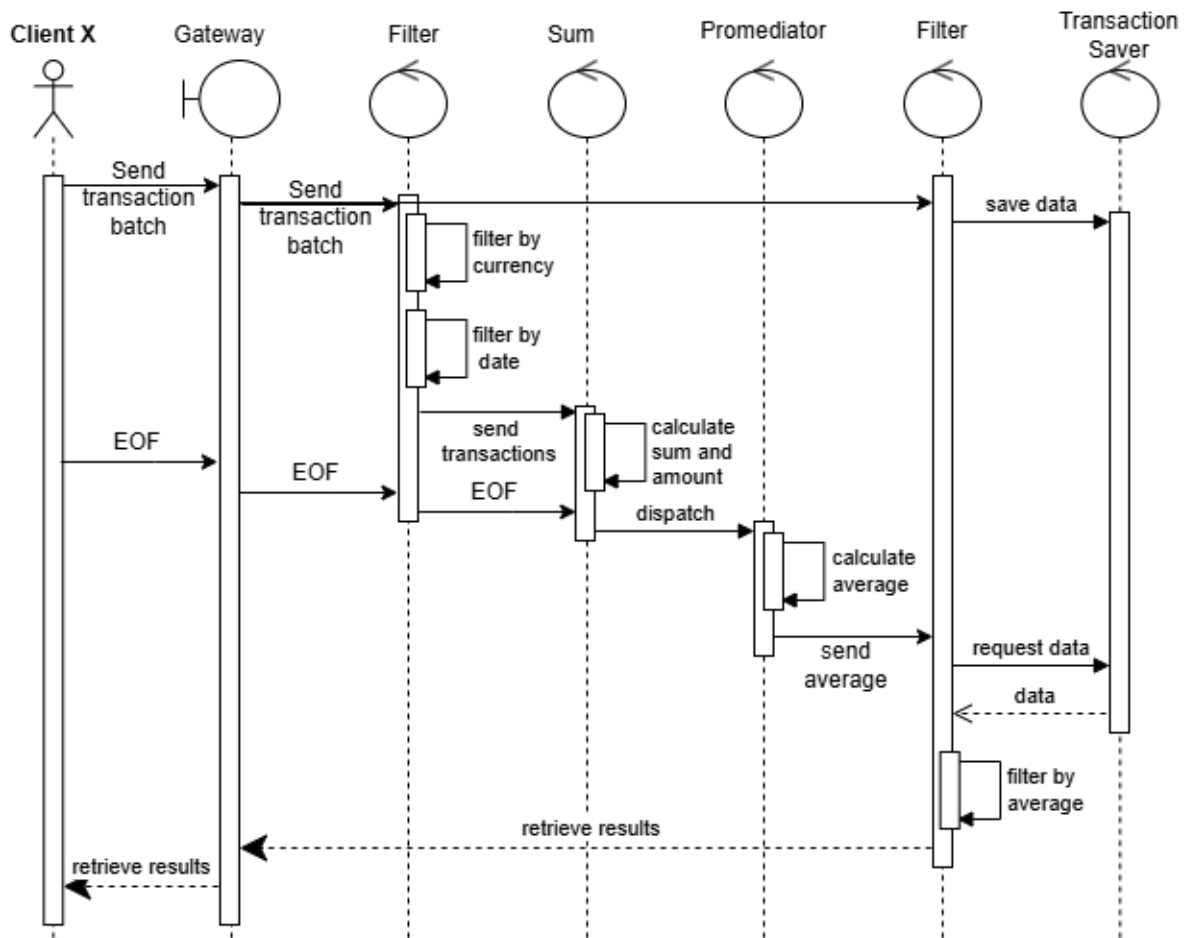


Figura 4.2: Diagrama de secuencia que ilustra la secuencia de operaciones a llevar a cabo para la resolución de la consulta 3.

Consulta 4

Analicemos ahora el caso 4, donde se requiere de identificar un patrón en las transacciones de forma que se debe identificar las cuentas de origen que cumplan con el patrón scatter gather con solo una cuenta intermedia.

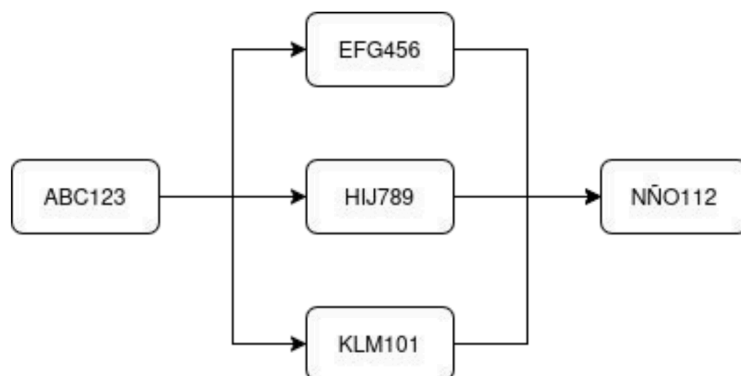


Figura 4.3: Ilustración del patrón scatter-gather como un grafo dirigido.

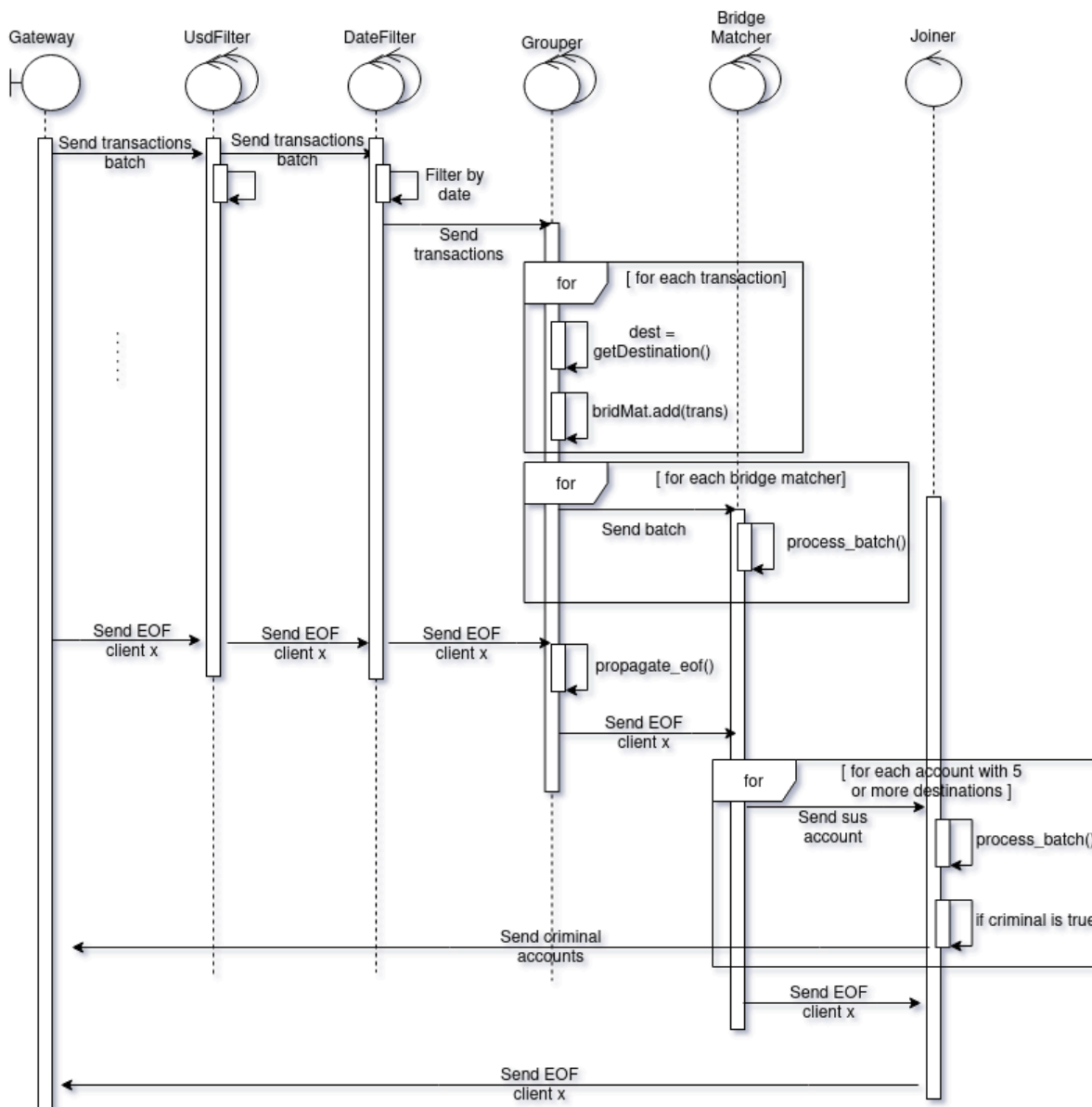


Figura 4.4: Diagrama de secuencia que ilustra la secuencia de operaciones a llevar a cabo para la resolución de la consulta 4.

Para la resolución de esta consulta se plantea implementar una estrategia de particionamiento (sharding) basada en la cuenta de origen para garantizar el determinismo en la distribución de carga, no solo con el objetivo de distribuir la carga, sino también permitiendo definir de forma determinista el mismo nodo de destino para las transacciones con mismo origen, simplificando así la lógica necesaria para sincronizar la información. Solo cuando se detecta un posible sospechoso las diferentes copias de “group” se comunican entre sí para notificar las posibles cuentas sospechosas de ser puentes entre el origen, y destino, sospechosos de lavado de activos.

Flujo del sistema

Veamos ahora las diferentes actividades desarrolladas por los diferentes actores del sistema en diferentes escenarios.

Punto de Entrada

En principio, observamos el flujo entre los nodos en el punto de entrada al sistema para así comprender mejor las actividades desarrolladas previo a la resolución de las consultas.

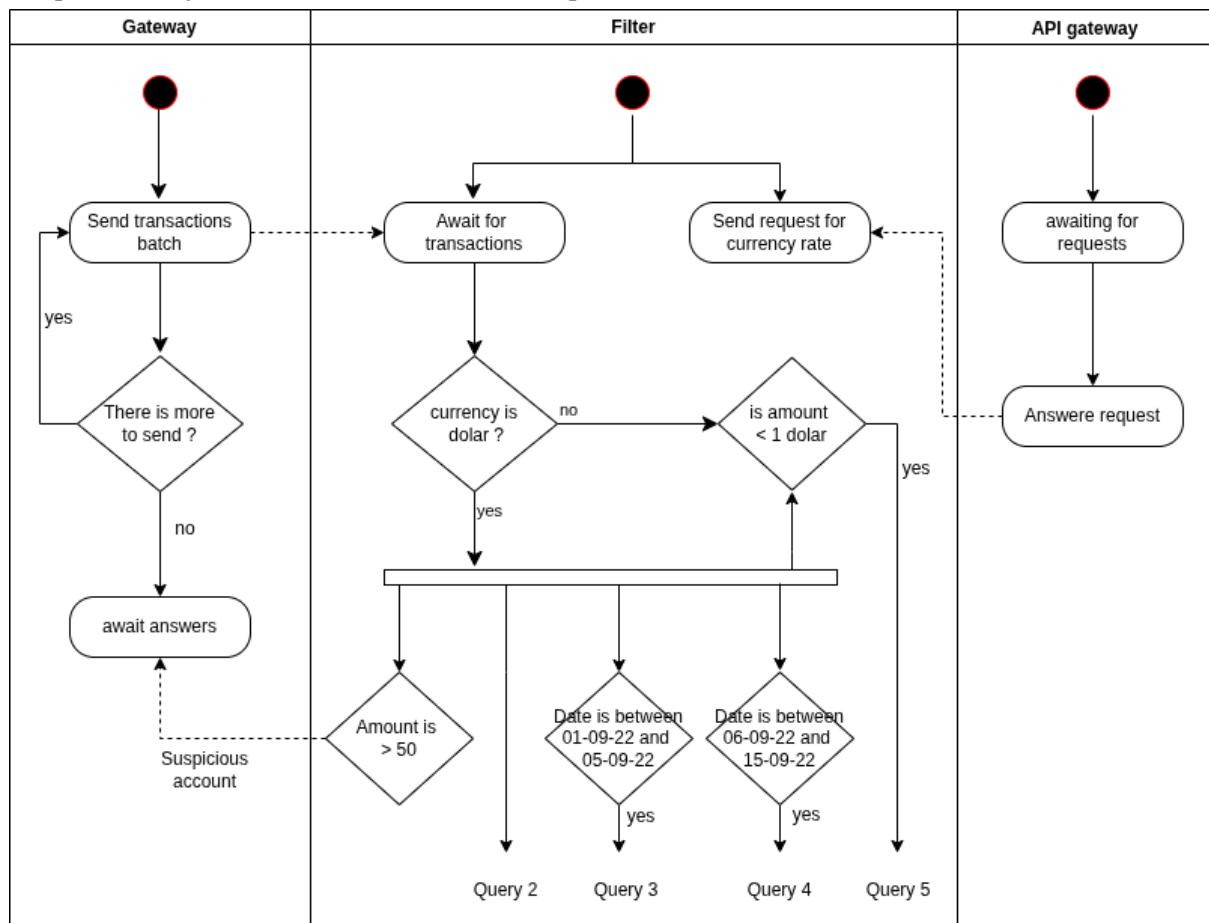


Figura 4.5: Diagrama de actividades indicando las actividades desarrolladas en el punto de entrada al sistema propuesto.

Mientras el gateway recibe los batches de transferencias del usuario este los deriva al filter quien se encargará de distribuir las transferencias, en la medida que estas cumplan las condiciones asociadas, a las diferentes queues donde los workers correspondientes se encargaran de procesarlas.

Consulta 3

Observemos ahora el caso 3 donde contamos con dos flujos de transferencias divididos a ser procesados para resolver una única consulta.

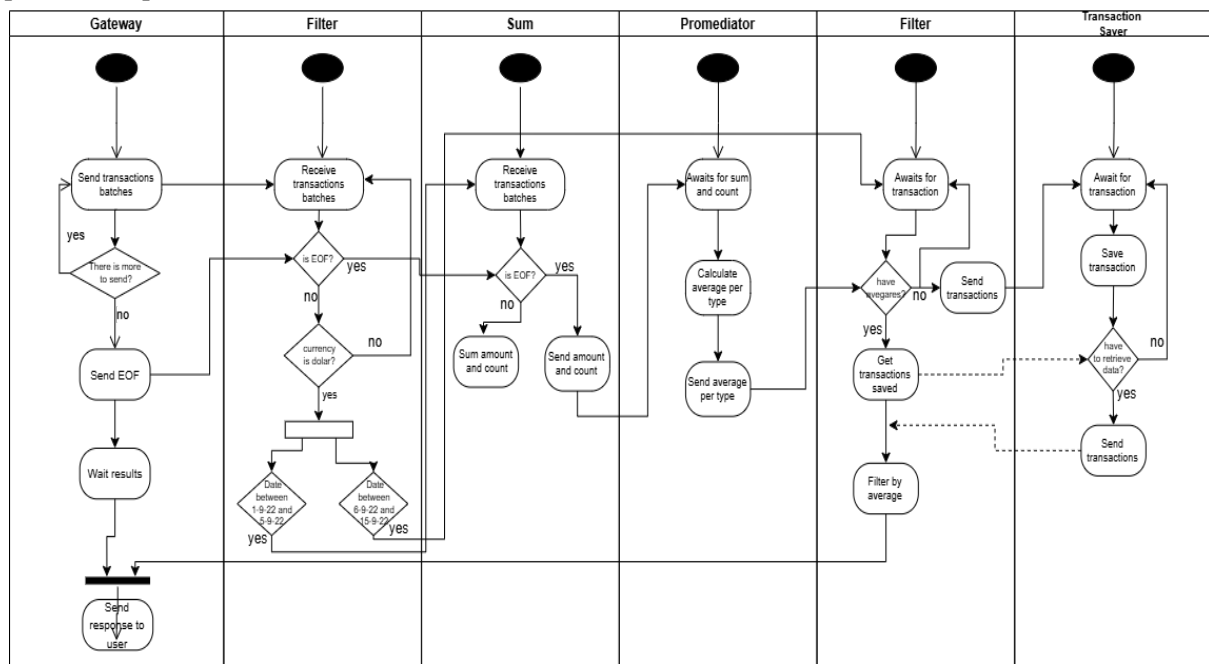


Figura 4.6: Diagrama de actividades en seguimiento del desarrollo de la consulta 3.

Del diagrama podemos observar que en una primera fase, el primer filter se encarga de dividir las transferencias en base a la fecha de realización. De esta manera, hasta que los nodos de Sum no reciben el EOF por parte del cliente estos calculan el amount y el count para los distintos tipos de transferencias.

Cuando reciben el EOF, le envían sus resultados parciales al Promediator. Una vez calculado el promedio el nodo correspondiente notifica a la entidad encargada de procesar los posibles casos de lavado.

El filter encargado le pide al transaction saver que le devuelva, si es que tiene, las transferencias que fue guardando al no poder procesar sin el promedio. Este procede a diferenciar los casos de lavado ya sea desde transacciones que le pueden llegar desde el filter original o las persistidas.

Al finalizar le envía los resultados al gateway para devolverle al usuario los resultados.

Consulta 4

En optimización a la primera propuesta planteada se ha implementado, para la resolución de la query 4 un sistema de workers “group” encargados de rutear las transacciones a un “Bridge Matcher” determinado (mediante sharding en base al origen de la transacción) para que este lleve conteo de las transacciones enviadas por cada cuenta y notifique a los workers de la siguiente etapa sobre la existencia de esta posible cuenta sospechosa y posteriormente envíe toda la información al join. Para indicar la finalización todos los nodos group deben enviar el mensaje EOR a cada uno de los “Bridge Matchers” con el objetivo de dar por finalizada la recepción de transacciones por determinado usuario. En la fase de “Bridge Matchers” estos se notifican entre sí mediante un 2 fase commit con motivo de confirmar la finalización del análisis de las transacciones (esto se cumple dado el prefetch 1 explicado en las observaciones)

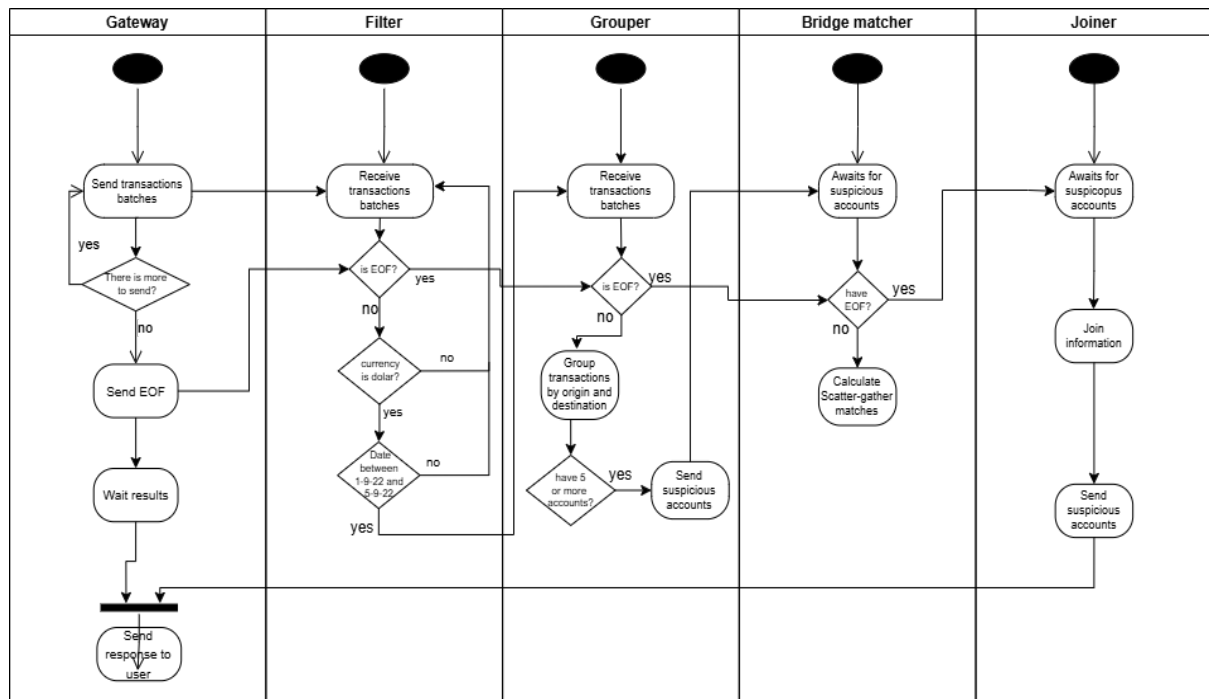


Figura 4.7: Diagrama de actividades del desarrollo de la consulta 4.

Organización Modular y Estructura

Diagrama de paquetes

Se muestran los diagramas de la estructura del sistema con respecto a sus módulos y paquetes, como así las dependencias entre ellas.

El paquete common centraliza las definiciones compartidas por todos los nodos, garantizando que tanto el Gateway como los Workers interpreten el formato de las transacciones, como persisten (data_saver) y como maneja los mensajes según cada worker (worker_handle)

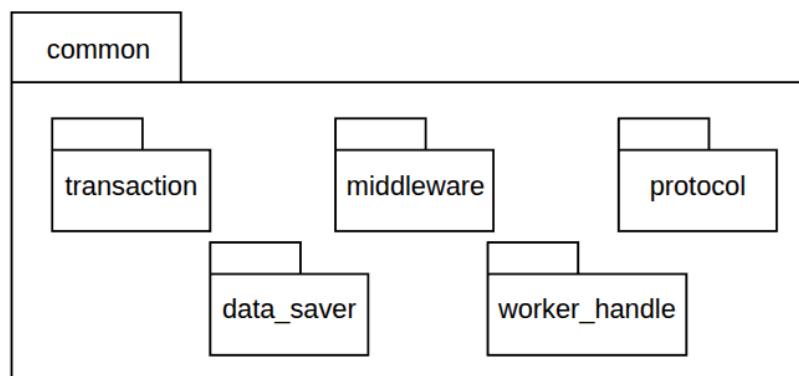


Figura 5.1: Paquete que tienen en común los demás

Los módulos de los workers van a ser muy parecidos entre sí, tal así que se lo puede agrupar de la siguiente manera:

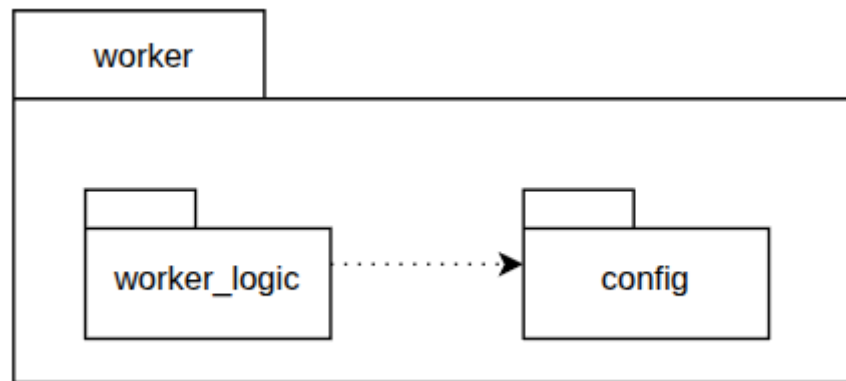


Figura 5.2: Paquete del worker

Con worker nos referimos a toda instancia replicable del sistema que ejecuta una tarea de cómputo específica, como el Group, Bridge Matcher, Sum, Counter, Transaction Saver, Filter, Join.

A continuación, se detalla la estructura que van a tener el Gateway y el Filter.

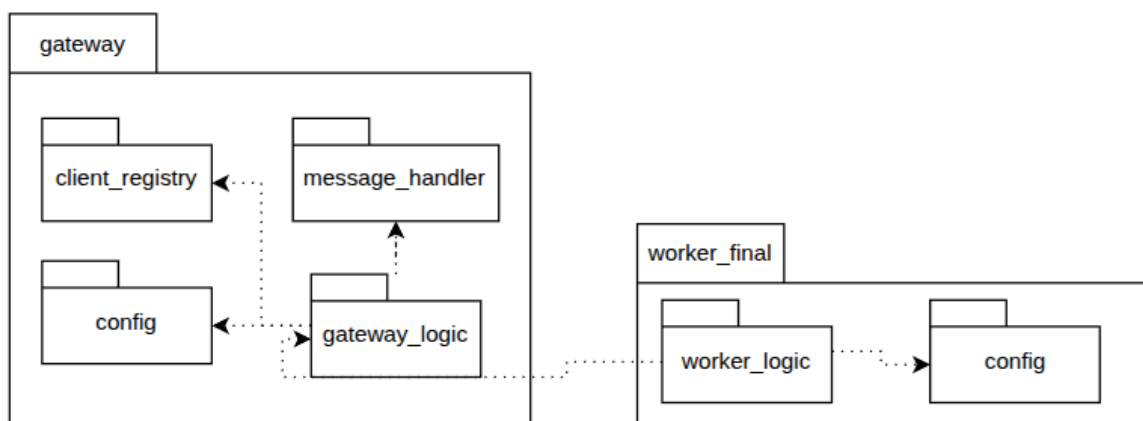


Figura 5.3: Paquete del gateway y del worker “final”

- Gateway: Funciona como el punto de enlace (interfaz) del sistema. Se encarga de la gestión de clientes (client_registry) y del orquestamiento de mensajes hacia los filtros mediante gateway_logic.
- Worker final: Es la instancia final de cada flujo de las queries que se encarga de enviarle los resultados al gateway, como los filters por monto (Q1 y Q3), join (Q4 y Q2) y counter (Q5). Estos se encuentran al final del flujo de cada query.

Notar que cada paquete cuenta con una config independiente. Eso permite que cada nodo gestione sus propios parámetros de red y variables de entorno de forma aislada.

Infraestructura y Despliegue

A continuación se detalla la infraestructura ideada para el funcionamiento distribuido del sistema.

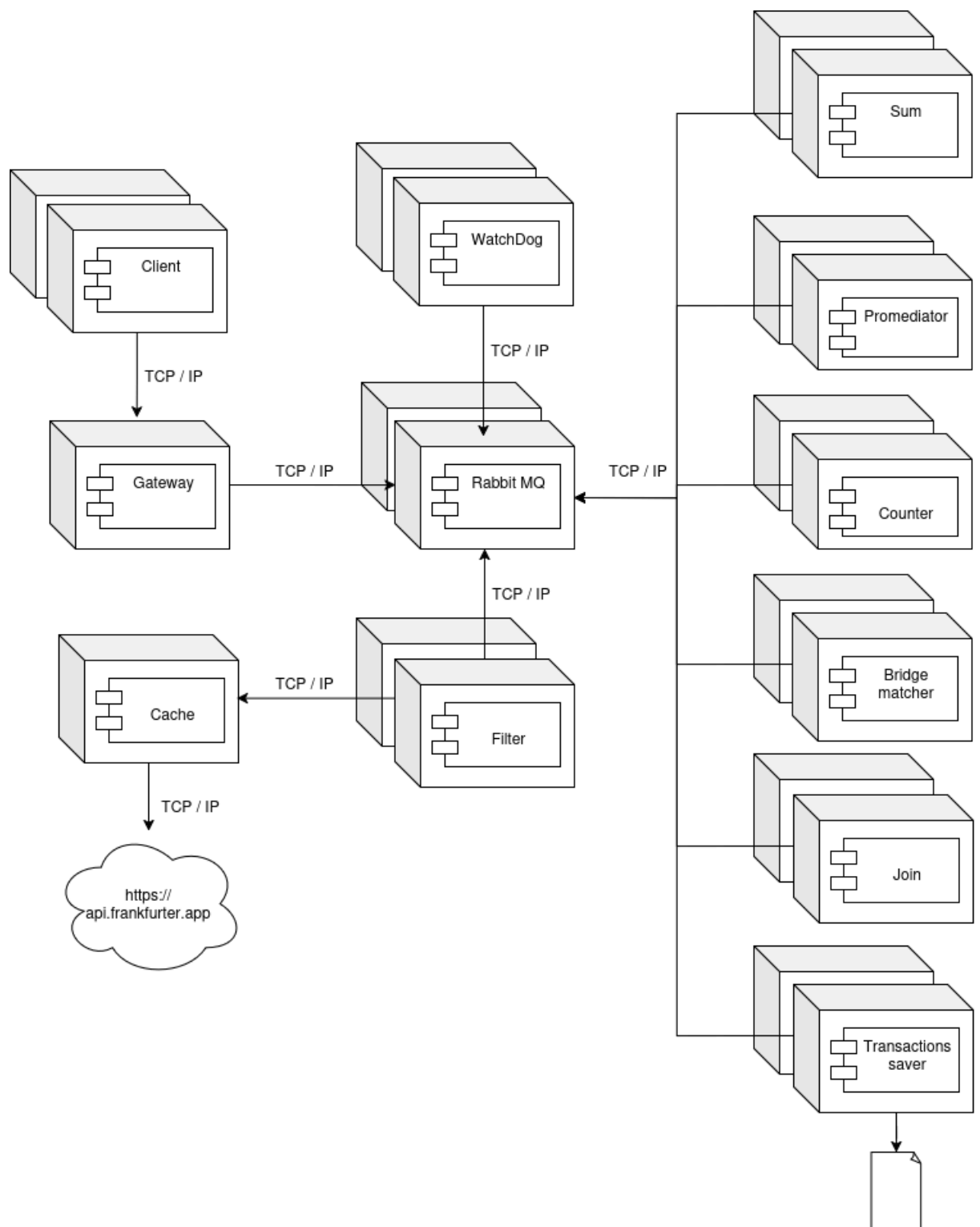


Figura 6.1: Diagrama de despliegue del sistema propuesto.

Características de la arquitectura:

- Para la comunicación entre nodos internos del sistema, se definen las queues de RabbitMQ como medio de comunicación y distribución de trabajo como también como medio de notificación general mediante broadcast según lo requiera la lógica de sincronización entre nodos.

- El filtrado de datos, entiéndase reducción de columnas o resolución de consultas básicas, es realizado por el filtro en el ingreso al sistema, de esta manera se reduce la magnitud de los datos circulantes y se da una respuesta temprana a la primer consulta, reduciendo el tiempo ocioso del cliente y del gateway.

Tolerancia a fallos problemáticas y soluciones

El sistema puede sufrir caídas repentinas en varios de sus nodos, por lo cual no se requiere que todos los nodos estén activos en simultáneo en todo momento.

Para el desarrollo de la tolerancia a fallos del sistema, se consideraron las siguientes problemáticas y soluciones:

Pérdida de datos ante posibles caídas

Dado que se plantea la posible caída de workers se propusieron las siguientes situaciones dependiendo de las características del worker.

- **Nodos stateless:** Incluye nodos como USD Filter, Date Filter, Amount Filter 1, Q5 Date Filter, Sum y Payment Format Filter, Group. En algunos casos es necesario persistir el manejo de EOF que lleva por cliente para comunicárselo a la capa posterior, pero en general todos se comportan de manera similar.
 - Si el nodo crashea luego de leer un mensaje o durante su procesamiento, el mensaje no recibe el ACK y se reencola automáticamente para ser procesado por otra instancia.
 - Si el nodo crashea justo después de enviar su resultado pero antes de hacer ACK, se reencolará el mensaje original, provocando un duplicado. Se ha determinado delegar la resolución de estos duplicados a las capas posteriores.
- **Nodos stateful:** Incluye nodos como Counter, Currencies Cache, Bridge Matcher, Join, Promediator, Amount Filter 3 y Transactions Saver.
 - Estos nodos deben persistir su progreso para no perder cálculos intermedios ni generar inconsistencias al reiniciar.

Identificación y Manejo de duplicados

Para asegurar la idempotencia del sistema frente a los mensajes duplicados, se implementó un mecanismo de deduplicación usando un ID por batch.

- **Generacion de IDs:** Los IDs de cada batch se calculan utilizando una función de hash basada en su contenido.
- **Deduplicador (sliding window):** Para evitar procesar un batch repetido, se mantiene un historial de procesados. Se utilizan dos estructuras de datos:
 - Un Set para búsquedas de duplicados
 - Una Queue para mantener el orden de llegada temporal. Cuando la capacidad máxima se alcanza, se extrae el ID más antiguo de la cola y se borra del Set.

Este deduplicador se usan solo en aquellos nodos que no son idempotentes, como es el caso del Promediator y Gateway.

Persistencia de datos y Recuperación

Para evitar pérdidas de datos ante cortes abruptos, los nodos ejecutan una persistencia mediante la escritura de logs y Checkpoints:

- **Escritura atómica:** Las operaciones se escriben inicialmente en archivos temporales y luego se renombran de forma atómica, evitando la corrupción de datos ante cortes durante la escritura.
- **Estructura del Log:** Cada mensaje procesado genera una entrada en el WAL. Cada entrada incluye un tipo (Log o Checkpoint) y su contenido asociado (el mensaje procesado en el caso de un Log, o el estado completo del nodo en el caso de un Checkpoint)
- **Checkpoints y restauración:** Periódicamente se almacena un checkpoint con el estado interno del nodo. Este checkpoint se guarda en el mismo archivo que el WAL, reemplazando los registros anteriores. Ante la recuperación de un nodo, primero se carga el checkpoint (ubicado al comienzo del archivo) y luego se reproducen las entradas de log registradas después del checkpoint, restaurando así el estado más reciente del sistema.

WatchDog y Algoritmo de elección (Bully)

Se incorporó un sistema de monitoreo (WatchDog) para detectar y reanimar automáticamente contenedores caídos.

- Los workers observados envían un heartbeat recurrente al sistema WatchDog. Un hilo observador verifica internamente su recepción. Si transcurre un umbral de tiempo sin novedades, el worker se declara caído y se procede a su reinicio.
- El propio sistema posee réplicas. Para coordinar cual de las instancias ejecuta comandos de reinicio, se implementa el Algoritmo Bully:
 - Al iniciarse, las réplicas postulan su ID.
 - Si una réplica detecta la ausencia del líder, inicia una elección.
 - Aquel nodo que no recibe un ID superior al suyo se autoproclama Líder y lo notifica por broadcast.
 - Solo el Líder ejecuta la detección activa y reinicio, mientras los esclavos mantienen actualizados sus estados pasivos para tomar el control instantáneamente ante una eventual caída del nodo principal.

Consideraciones

- El sistema cuenta con una única instancia del nodo “Gateway” como entrada al sistema, siendo este un único punto de falla para el cual no se han considerado contramedidas aún.
- Para el desarrollo del trabajo práctico se ha hecho uso de la feature “prefetch”, particularidad del middleware “RabbitMQ”, con valor igual a uno, de forma que los trabajadores reciben los mensajes de a uno por vez, permitiendo recibir los mensajes en el orden en que fueron enviados al middleware simplificando la consideración del desorden y la complejidad de coordinar los múltiples nodos. Sin embargo, esta feature presenta la desventaja de reducir notablemente la velocidad a la que se reciben los mensajes.

Referencias

- [1] Altman, E. (s.f.). IBM Transactions for Anti Money Laundering (AML) [Dataset]. Kaggle. <https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>
- [2] Frankfurter. (s.f.). *Frankfurter API: Foreign exchange rates API*. <https://api.frankfurter.dev/>
- [3] pablotroca. (s.f.). Money laundering analysis [Notebook]. Kaggle. <https://www.kaggle.com/code/pablotroca/money-laundering-analysis>